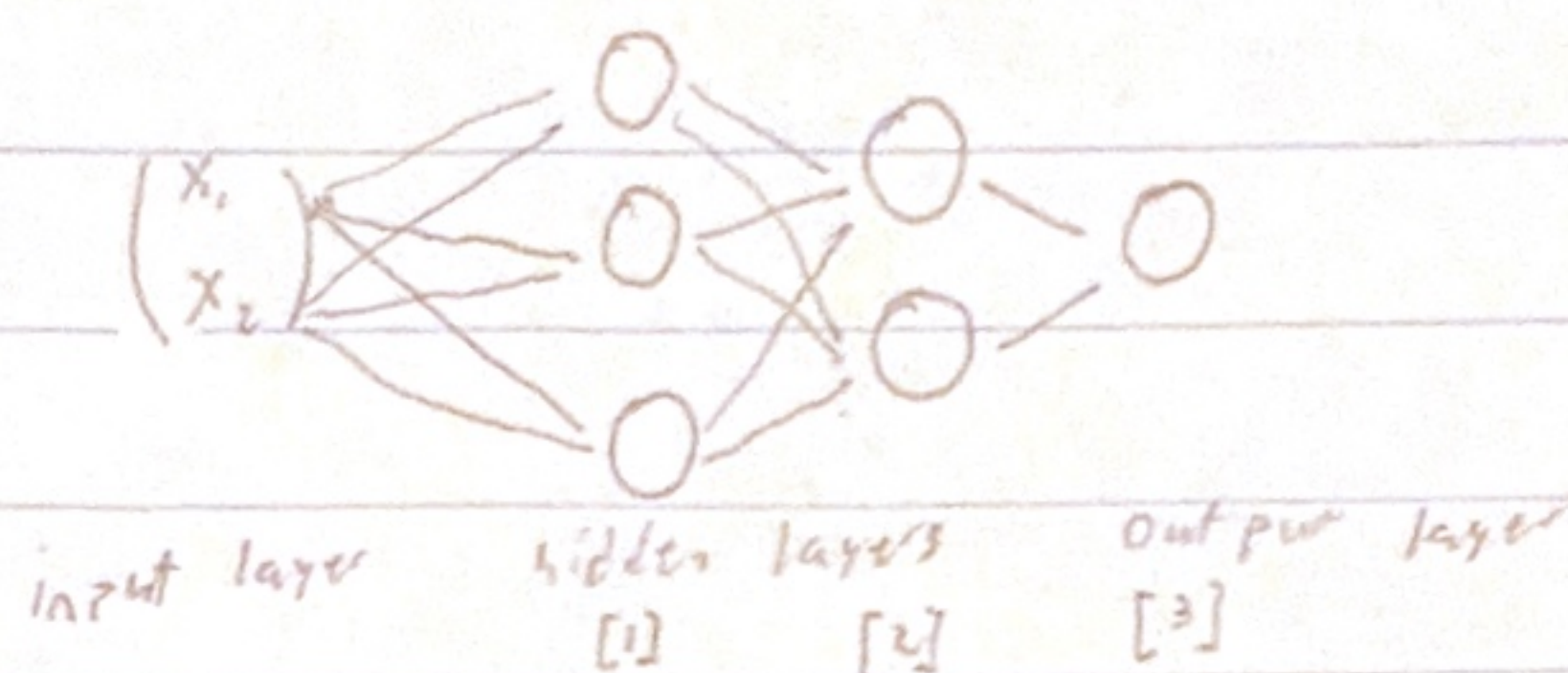


Neural Network

In this file, we will go through the necessary mathematical details to implement a shallow neural network, we will go through the architecture, forward propagation, and the backward propagation equations so that we can implement this neural network. Let's start with the architecture:



As we can see, we have an input layer that takes two features, which leads to our first hidden layer, with three neurons, which leads to our second hidden layer, with two neurons, which leads to our output layer, with one neuron.

Now that we have asserted our architecture, let's introduce the forward propagation equations. These are the equations that our feature vectors would go through for our model to make a prediction.

$$z^{[1]} = w^{[1]}x + b^{[1]} \quad \text{shapes: } z^{[1]}: (3,1), w^{[1]}: (3,2), x: (2,1), b^{[1]}: (3,1)$$

$$a^{[1]} = \sigma(z^{[1]}) \quad a^{[1]}: (3,1)$$

$$z^{[2]} = w^{[2]}a^{[1]} + b^{[2]} \quad z^{[2]}: (2,1), w^{[2]}: (2,3), b^{[2]}: (2,1)$$

$$a^{[2]} = \sigma(z^{[2]}) \quad a^{[2]}: (2,1)$$

$$z^{[3]} = w^{[3]}a^{[2]} + b^{[3]} \quad z^{[3]}: (1,1), w^{[3]}: (1,2), b^{[3]}: (1,1)$$

$$\text{Hypothesis Function } \hat{y} = a^{[3]} = \sigma(z^{[3]}) \quad a^{[3]}: (1,1)$$

Now that we have our forward propagation equations, we need to derive the backward propagation equations. Forward propagation makes the predictions while backward propagation derives the gradients of our loss with respect to each weight and bias tensor so that we can update them.

To go through backward propagation, we first need to talk about our loss function so that we can compute derivatives. We will use the same Binary Cross-Entropy loss as we did in Logistic Regression,

$$L = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log (1-\hat{y}^{(i)})$$

To derive gradients, we will compute derivatives of our loss function with respect to each of our 6 weight and bias tensors ($w^{[3]}, w^{[2]}, w^{[1]}, b^{[3]}, b^{[2]}, b^{[1]}$) and we will arrive at our parameter updates equations via gradient descent (backward propagation refers to this process of computing gradients).

Let's compute the derivatives:

$$w^{[3]}: \quad \frac{\partial L}{\partial w^{[3]}} = \frac{\partial L}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial w^{[3]}}$$

$$\frac{\partial L}{\partial a^{[3]}} = -\frac{1}{m} \sum_{i=1}^m \frac{y^{(i)}}{a^{[3](i)}} + \frac{(1-y^{(i)})}{(1-a^{[3](i)})}$$

$$\frac{\partial a^{[3]}}{\partial z^{[3]}} = a^{[3](i)} (1-a^{[3](i)})$$

$$\frac{\partial z^{[3]}}{\partial w^{[3]}} = \frac{\partial}{\partial w^{[3]}} (w^{[3]} a^{[2]} + b^{[3]}) = a^{[2]T} \quad \leftarrow \text{(Since the shape of the gradient must match the shape of the variable you are taking it with respect to)}$$

$$\frac{\partial L}{\partial w^{[3]}} = \left(-\frac{1}{m} \sum_{i=1}^m \frac{y^{(i)}}{a^{[3](i)}} + \frac{(1-y^{(i)})}{(1-a^{[3](i)})} \right) (a^{[2](i)} (1-a^{[3](i)}) a^{[2]T}$$

$$= -\frac{1}{m} \sum_{i=1}^m y^{(i)} (1-a^{[3](i)}) a^{[2]T} + (1-y^{(i)}) a^{[3](i)} a^{[2]T}$$

$$= \left[\frac{1}{m} \sum_{i=1}^m (a^{[3](i)} - y^{(i)}) a^{[2](i)} \right] T$$

$$b^{[3]}: \quad \frac{\partial L}{\partial b^{[3]}} = \frac{\partial L}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial b^{[3]}} \quad \leftarrow \text{(already derived)} \quad \frac{\partial L}{\partial z^{[3]}} = \frac{1}{m} \sum_{i=1}^m (a^{[3](i)} - y^{(i)})$$

$$\frac{\partial z^{[3]}}{\partial b^{[3]}} = 1$$

$$\frac{\partial L}{\partial b^{[3]}} = \frac{1}{m} \sum_{i=1}^m (a^{[3](i)} - y^{(i)})$$

$$w^{[2]}: \quad \frac{\partial L}{\partial w^{[2]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

$$\frac{\partial z^{[2]}}{\partial a^{[2]}} = w^{[3]T}$$

$$\frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2](i)} (1-a^{[2](i)})$$

$$\frac{\partial z^{[2]}}{\partial w^{[2]}} = a^{[1]T} \quad \rightarrow \text{(this actually becomes a tensor of shape (2,2,3), but for our purposes, we will represent it as } a^{[1]T} \text{, the math still works)}$$

$$\frac{\partial L}{\partial w^{[2]}} = \left(\frac{1}{m} \sum_{i=1}^m (a^{[3](i)} - y^{(i)}) \right) w^{[3]T} (a^{[2](i)} (1-a^{[2](i)}) a^{[1]T}$$

The shapes don't match, so we will implement an element-wise product

$$\frac{\partial L}{\partial w^{[2]}} = \left(\frac{1}{m} \sum_{i=1}^m (a^{[3](i)} - y^{(i)}) \right) w^{[3]T} \odot (a^{[2](i)} (1-a^{[2](i)}) a^{[1]T}$$

$$\frac{\partial L}{\partial b^{[2]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial b^{[2]}} = 1$$

$$\frac{\partial L}{\partial b^{[2]}} = \frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) \odot a^{[2](i)} (1 - a^{[2](i)})$$

$$\frac{\partial L}{\partial W^{[2]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial W^{[2]}} = \frac{\partial L}{\partial z^{[2]}} \odot a^{[2](i)} (1 - a^{[2](i)})$$

$$\frac{\partial L}{\partial z^{[2]}} = W^{[2]}$$

$$\frac{\partial a^{[2](i)}}{\partial z^{[2](i)}} = a^{[2](i)} (1 - a^{[2](i)})$$

$$\frac{\partial L}{\partial W^{[2]}} = \frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) \odot a^{[2](i)} (1 - a^{[2](i)}) \odot W^{[2]T}$$

$$\frac{\partial L}{\partial W^{[2]}} = \frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) \odot a^{[2](i)} (1 - a^{[2](i)}) \odot W^{[2]T}$$

$$\frac{\partial L}{\partial W^{[2]}} = \frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) \odot a^{[2](i)} (1 - a^{[2](i)}) \odot W^{[2]T}$$

$$\frac{\partial L}{\partial b^{[1]}} = \frac{\partial L}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial b^{[1]}} = 1$$

$$\frac{\partial L}{\partial z^{[1]}} = 1$$

$$\frac{\partial L}{\partial b^{[1]}} = \frac{1}{m} \sum_{i=1}^m (a^{[1](i)} - y^{(i)}) \odot a^{[1](i)} (1 - a^{[1](i)})$$

Putting everything together, we have the following (batch) gradient descent update formulas:

$$W^{[3]} := W^{[3]} - \alpha \left(\frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) a^{[2](i)} \right)^T$$

$$b^{[3]} := b^{[3]} - \alpha \left(\frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) \right)$$

$$W^{[2]} := W^{[2]} - \alpha \left(\frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) W^{[3]T} \odot a^{[2](i)} (1 - a^{[2](i)}) \right)^T$$

$$b^{[2]} := b^{[2]} - \alpha \left(\frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) \odot a^{[2](i)} (1 - a^{[2](i)}) \right)$$

$$W^{[1]} := W^{[1]} - \alpha \left(\frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) (W^{[2]T} (W^{[3]T} \odot a^{[2](i)} (1 - a^{[2](i)}) \odot a^{[1](i)} (1 - a^{[1](i)}) x^{(i)T}) \right)^T$$

$$b^{[1]} := b^{[1]} - \alpha \left(\frac{1}{m} \sum_{i=1}^m (a^{[2](i)} - y^{(i)}) (W^{[2]T} (W^{[3]T} \odot a^{[2](i)} (1 - a^{[2](i)}) \odot a^{[1](i)} (1 - a^{[1](i)}) \right)$$

(where α is the learning rate, usually set at 0.01)

we can simplify these formulas even more to obtain a vectorized implementation of batch gradient descent. Note that the shapes of the a 's and z 's derived in the forward pass will also change as we input a batch of examples, (shape of x is $(2, m)$.)

Let's go through the forward pass again to visualize the shapes

$$z^{[1]} = W^{[1]}x + b^{[1]} \quad \text{shapes: } z^{[1]}: (3, m), w^{[1]}: (3, 2), x: (2, m), b^{[1]}: (3, 1)$$

$$a^{[1]} = \sigma(z^{[1]}) \quad a^{[1]}: (3, m)$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \quad z^{[2]}: (2, m), w^{[2]}: (2, 3), b^{[2]}: (2, 1)$$

$$a^{[2]} = \sigma(z^{[2]}) \quad a^{[2]}: (2, m)$$

$$z^{[3]} = W^{[3]}a^{[2]} + b^{[3]} \quad z^{[3]}: (1, m), w^{[3]}: (1, 2), b^{[3]}: (1, 1)$$

$$a^{[3]} = \sigma(z^{[3]}) \quad a^{[3]}: (1, m)$$

Note that the b 's will broadcast to become a matrix with m columns, which are copies of the first column.

Now we can simplify our previously derived formulas:

$$W^{[3]} := W^{[3]} - \frac{\alpha}{m} (a^{[3]} - y) a^{[2]T}$$

$$b^{[3]} := b^{[3]} - \frac{\alpha}{m} \sum_{i=1}^m (a^{[3](i)} - y^{(i)})$$

$$W^{[2]} := W^{[2]} - \frac{\alpha}{m} W^{[3]T} (a^{[2]} - y) \odot a^{[1]} (1 - a^{[1]}) a^{[1]T}$$

$$b^{[2]} := b^{[2]} - \frac{\alpha}{m} \sum_{i=1}^m (a^{[3](i)} - y^{(i)}) W^{[3]T} \odot a^{[2](i)} (1 - a^{[2](i)})$$

$$W^{[1]} := W^{[1]} - \frac{\alpha}{m} \left(W^{[2]T} (W^{[3]T} (a^{[3]} - y) \odot a^{[2]} (1 - a^{[2]})) \odot a^{[1]} (1 - a^{[1]}) x^T \right)$$

$$b^{[1]} := b^{[1]} - \frac{\alpha}{m} \sum_{i=1}^m W^{[2]T} (W^{[3]T} (a^{[3](i)} - y^{(i)}) \odot a^{[2](i)} (1 - a^{[2](i)})) \odot a^{[1](i)} (1 - a^{[1](i)})$$

Now we can simply implement these in a training loop, allowing our model to learn.